Governors State University
OPUS Open Portal to University Scholarship

All Capstone Projects | Student Capstone Projects

Fall 2023

Robotics Drone Integration

Sridevi Kandula

Follow this and additional works at: https://opus.govst.edu/capstones

For more information about the academic degree, extended learning, and certificate programs of Governors State University, go to http://www.govst.edu/Academics/Degree_Programs_and_Certifications/

Visit the Governors State Computer Science Department
This Capstone Project is brought to you for free and open access by the Student Capstone Projects at OPUS Open Portal to University Scholarship. It has been accepted for inclusion in All Capstone Projects by an authorized administrator of OPUS Open Portal to University Scholarship. For more information, please contact opus@govst.edu.

Robotics Drone Integration

By

Sridevi Kandula
Btech, Andhra Loyola Institution of Engineering and Technology, 2021

GRADUATE CAPSTONE SEMINAR PROJECT

Submitted in partial fulfillment of the requirements

For the Degree of Master of Science,

With a Major in Computer Science

Governors State University
University Park, IL 60484

2024

Abstract

Robotics is a combination of Engineering and Computer Science, the study of the design, upkeep, application, and use of robots. Robotics seeks to develop tools that help humans increase efficiency and precision while reducing errors. In healthcare, robots enable minimally invasive surgeries and assist in patient care. Robots are sent to distant planets and moons to gather data. Beyond these applications, robots aid in search and rescue missions, contribute to education and research, and serve in numerous other capacities, showcasing their versatility and significance in modern society.

The objective of this project is to build a control system and software system for a robot that will allow drive autonomously. The robot will be able to calculate the distance, identify the color, pick up objects, and know its precise location in the game field by using a wide range of sensors. A mechanical arm is used to perform certain tasks and can lift itself up, it can also hold objects without releasing them until instructed. The robot's arm should have the flexibility to be able to shoot a drone at the end as far as possible to obtain the highest possible points. The robot will be able to carry a drone in its compartment and not drop it until the end when it is released. When the tasks are at a completion stage the robot should be capable of relocating to its Destin destination where the robot is to be when completed.

The technologies we are going to use our Android Studio to program and use opModes that instruct the robot on its task. We use the hub, which is the main processing unit that communicates with the controller and stores the code that will instruct the robot. Optical Studio Sensors in the robot are used to identify the color that is needed to find certain locations and objects. The Ultrasonic Sensor MaxBotix 12C is to be used in robotics to measure the distance between the robot and the object in the game field. This will aid the robot to avoid collapsing into other items or robots. The use of an IMU is important to control the speed and its direction. Using its sensors properly will avoid improper malfunctions.

Table of Content

1.1 Competitive Information – Team Overview ..... **Error! Bookmark not defined.**
2.1 Strategy - Relationship to Other Applications/Projects ..... **Error! Bookmark not defined.**
3. Project Technical Description - Robot Control Station ..... **Error! Bookmark not defined.**
3.1Application Architecture – Robot Design ..... **Error! Bookmark not defined.**
4. Application Information flows - Hardware ..... **Error! Bookmark not defined.**
Figure:4.3 MinibotHardware.java image 3 ..... **Error! Bookmark not defined.**
4.1 Application Information flows - Autonomous phase ..... **Error! Bookmark not defined.**
In the autonomous phase of a match, the robot operates without any human input or control. **Error! Bookmark not defined.**
5. Capabilities ..... **Error! Bookmark not defined.**
7. Requirements ..... **Error! Bookmark not defined.**
8. Open Issues ..... **Error! Bookmark not defined.**
9. Acknowledgements ..... **Error! Bookmark not defined.**
10. References (10) ..... **Error! Bookmark not defined.**

1.Project Description

The objective of this project is to build a control system and software system for a robot that will allow drive autonomously. The robot will be able to calculate the distance, identify the color, pick up objects, and know precise locations in the game field by using a wide range of sensors. A mechanical arm is used to perform certain tasks and can lift itself up, it can also hold objects without releasing them until instructed. The robot's arm should have the flexibility to be able to shoot a drone at the end as far as possible to obtain the highest possible points. The robot will be able to carry a drone in its compartment and not drop it until the end when it is released. When the tasks are at a completion stage the robot should be capable of relocating to its Destin destination where the robot is to be when completed.

1.1 Competitive Information – Team Overview

Team One consists of Sandra Garcia, Sai Vidula Boppana, Chandana Mandadi, Sridevi Kandula, and Yashwanth Reddy Alavala. Team One is working on the robot project from Governor State University as part of the CPSC 8985-03 course. Our field area is located in the university library where we meet to enhance our robot, named "R2D2", to meet the competition requirements. Team 1 represents Red Alliance Partner 1 and occupies a portion of the field in the backstage area. We named our mini robot "R2D2". The specific robot control hub is named "Y55D", which can be found and connected through a Wi-Fi network.

2. Applications - Relationship to Other Applications/Project

Team One began to read the information available on the FTC website related to the competition on the website (https://ftc-_docs.firstinspires.org/en/latest/index.html). The first step was to review the information provided and download the following applications:

- Java Script code was installed in the laptops; this is a programming language required and compatible to program the robot controller using the Android studio environment. Software Development Kit (SDK) tools used are both served by JavaScript. We use Chrome when installing.
- Android Studio was downloaded and installed on our laptop devices by accessing the links on the website(https://developer.android.com/studio.) Team One proceeded by following the tutorial guidelines and requirements. Two laptops are MacOS, and they took additional steps but have successfully installed Java. Android Studio is one of the integrated development environments that is required by the competition guidelines.
- The SDK zip file for the competition was downloaded from the GitHub repository website provided on the FTC website (https://github.com/FIRST-techChallenge/FtcRobotController) The newest release of v9.0 was downloaded and unzipped. The file was then uploaded to Android Studio. The team code file was created so that Team One maintains the Java file of the robot. The top samples are not to be modified but can be copied into another file under the team code so that the new file can be modified.
- Git Hub – We have downloaded GitHub Desktop and each team member created accounts so that it is easier to share code files and modify any necessary parts. The below steps were then followed to clone the repository from team leader Sandra's account.
- Each team member created a Git Hub account and downloaded the Desktop app in Figure 2-1.

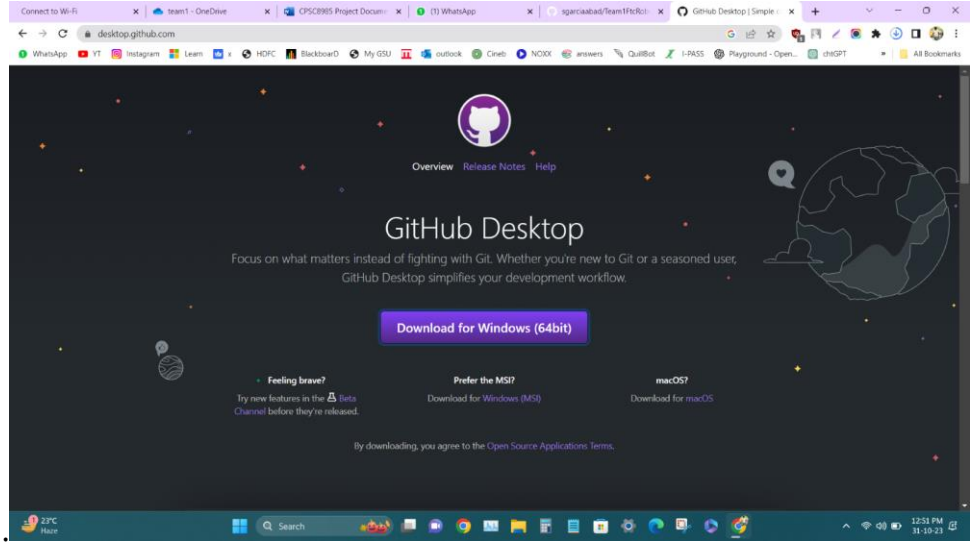


Figure 2.1 - GitHub Desktop

- Each member continued to Sign in and configure, see Figure 2.2.

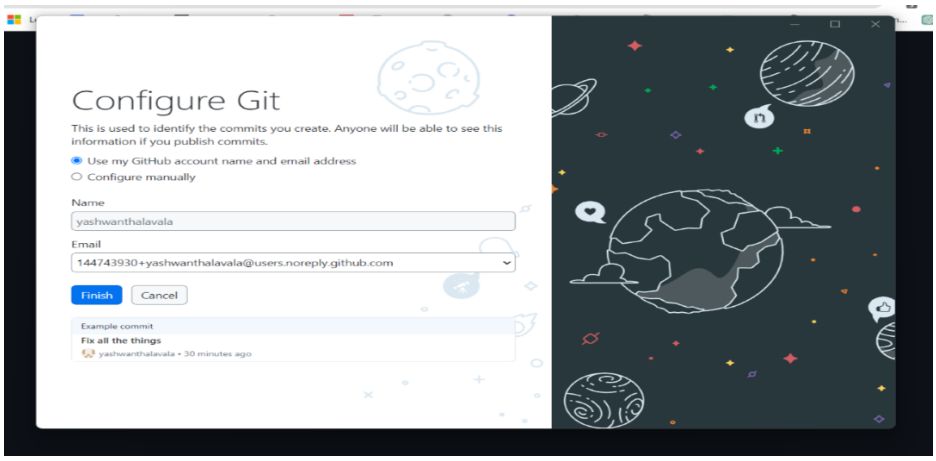


Figure 2.2 GitHub Configure

- Each team member took Repository access from Sandra by providing individual userID to be granted access.
- The team member then cloned the repository – this will allow each team member to contribute to develop the code.
- To clone the repository, select clone a repository from the internet, see Figure 2.3.

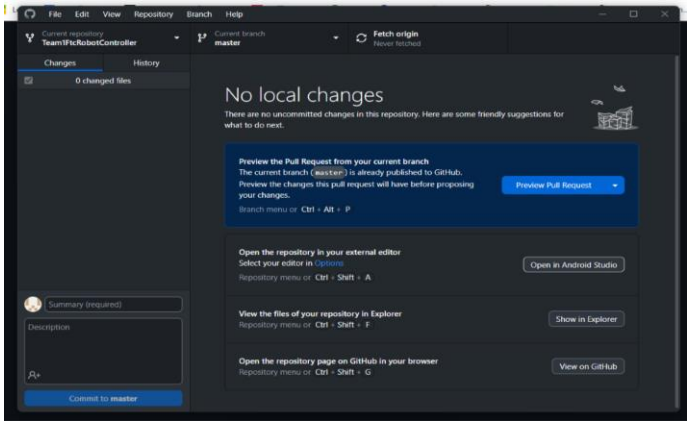


Figure 2.3 Clone

- After cloning open the team code.
- Create a new Branch to Add code and test without disturbing the main repository see Figure 2.4.

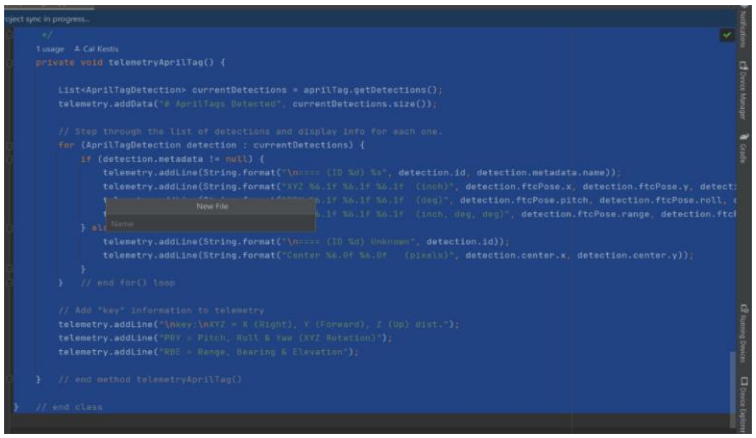


Figure 2.4 New Branch

1.2 2.1 Strategy - Relationship to Other Applications/Projects

The Teams were provided the reveal video for the competition to learn the rules of the game to strategies a plan to score the most points. We determined that our robot R2D2 has the following characters:

Fast Pace to accomplish the score needed to obtain the most points and get from one end to another in a fast time limit.
The robot is required to be 12” square feet according to the guidelines in the manuals. Team One was provided with a robot by our professor. The measurement of the robot will play an important role in the encoder portion of the code.
The robot requires a hand in which it is able to carry and hold pixels to designated areas. The arm extends to perform functions that are needed to pick up, drop, and hold a limit of two pixels.
Serviceable and robust if anything is wrong with the robot, we need to be able to correct it otherwise we can be disqualified. Since our robots are provided by our professor, we as a team are committed to treating robots with caution to avoid issues in competition.

The second stage of our meeting was to come up with a plan for what actions our robot could perform for the most points. We reviewed the video multiple times and used an image from the manual

simulating the field to draw out the performance. The next steps will be to review our steps with our alliance team to correct any interference the robots can have with each other.

2 3. Project Technical Description - Robot Control Hub

Robot “R2D2” is implemented by a Point-to-Point control system including a robot, REV control hub, Wi-Fi connection, driver station, and a Logitech game controller. The Robot Control Console is a local network created by the REV Control Hub to program and manage the device. The driver station is where we will use the software management and the FTC Driver station application that communicates with the Control Hub by using the Wi-Fi Name - Y55D. Figure 3.1 demonstrates the control station layout.

Figure 3.1 Robot “R2D2” Driver Station

2.1 3.1 Application Architecture – Robot Design

The control Console of Robot "R2D2” assigned to Team One includes the following hardware. A REV control hub is operated by pulling power from a slim 12V battery. The battery is connected to the power ports on the control hub. The control hub had an LED which notifies us of any connection status. When the LED is green the hub is ready for connection, LED will turn blue every 5 seconds indicating the hub is healthy. The hub comes with an orange USB and USB-C wire that is used to connect the hub with an A.S. programmed code file. The REV Control Hubs come with a default network and password REV Robotics advises that during competition teams utilize a 5 GHz channel for robot communication. REV control hub is a combination of the expansion hub and Android device built into one device. For safety reasons, the battery has a 20A fuse built in. A mechanical switch is used to turn on/turn off the power. The hub has 4 ports per hub numbered from 0-3 that connect with a 4-pin JST-PH style connector used to power the motor and is placed on the section labeled motor. A 4-pin JST-PH style connector is used for quadrature encoders used in tandem, with the motor adjacent. The connector is placed in the encoder label on the control hub. Servo Hub (servo is listed in orange on the hub followed by input 0-5) has 6 built-in servo ports. They accept standard 3-wire header-style connectors, ground pin is on the left side of the servo port. On Robot "R2D2” the servo is on input 0. The Touch Sensor is on the digital connector labeled 0-1. The sensor has 4 independent digital input/output (I/O) ports, two digital I/O pins for a total of 8 digital I/O pins, REV Robotics Touch sensor to one of the digital I/O REV Robotics. The touch sensor only needs to connect to one of the two available digital I/O pins, the second digital I/O pin in the port is the one that gets connected

when a standard REV Robotics 4-pin JST PH cable. A camera is connected to the control hub and the driver station. The camera is used to detect objects and April Tags. Figure 3.2 is a layout of the “R2D2” robot control hub.

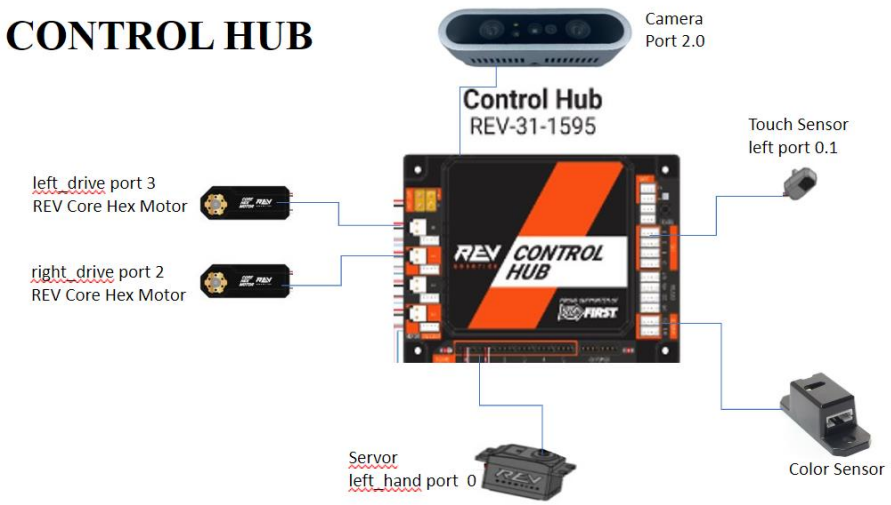
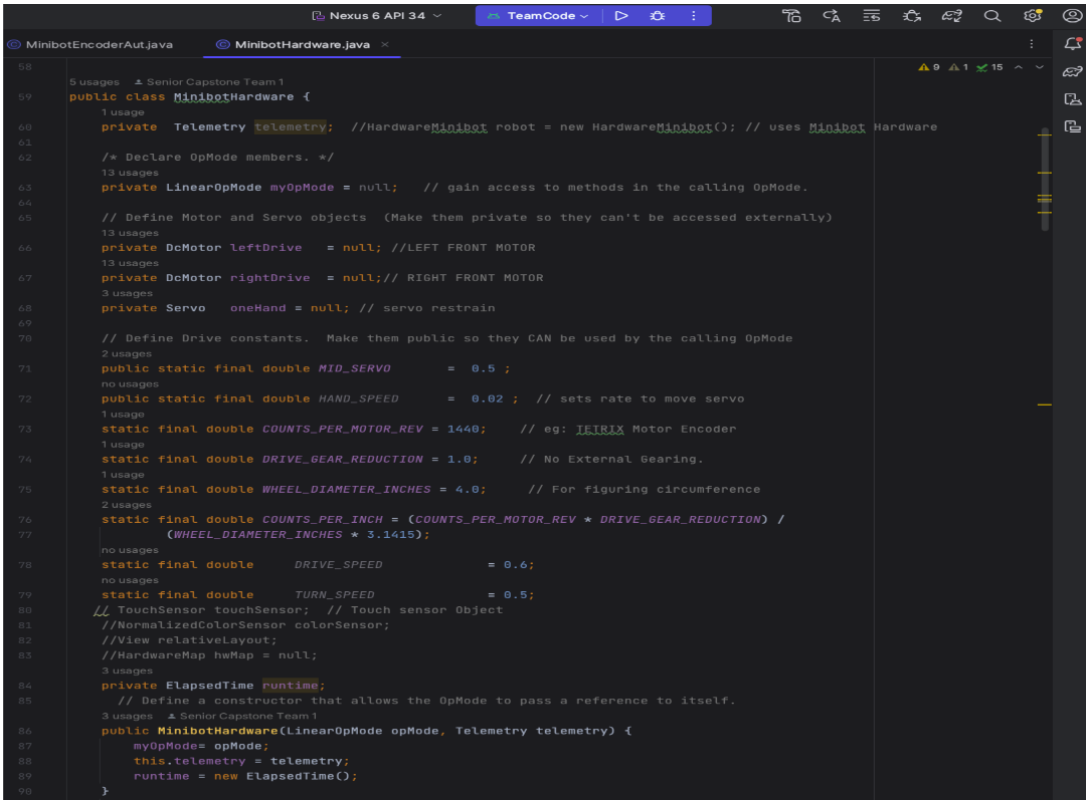


Figure 3.2 Control Hub

4. Application Information flows - Hardware

Team One decided to use the encoder sample code provided by FTC. Encoders are any sensors that track the rotational angle of the mechanism. These are commonly known as relative and absolute encoders; relatives are the most common. The output and input are relative to the power of the robot, information is lost between power cycles. In FTC the relative encoder sends information to the control hub and must be plugged into the encoder ports near the motor port to work. We use ticks or counts to refer to an encoder position. Every move is one tick from the starting angle. Then we can report the count per revolution to use this to convert them into degrees or revolutions.

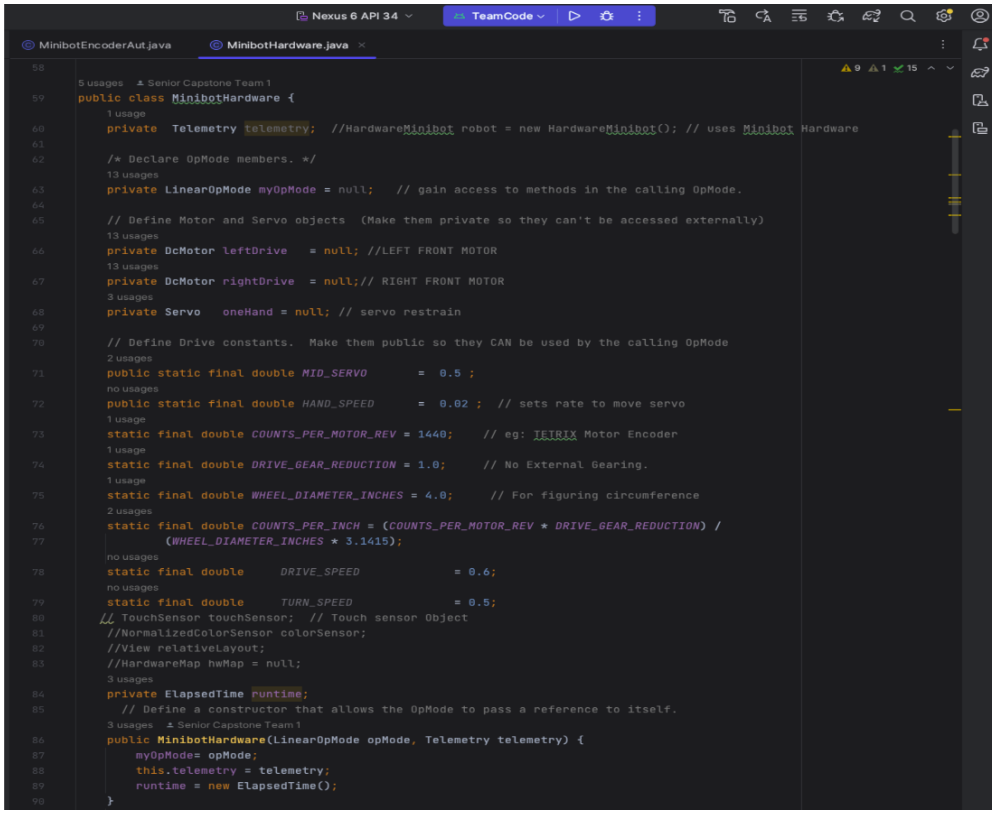
To start to accomplish movement in Robot “R2D2” Team One initiated a file in team code called Minibot Hardware.java. Figure 4.1 includes the initial declaration of the hardware code located in the team code file of the A.S. file directory. The code below starts with the class and declares left and right Drive, DC Motor, and Servo oneHand with the corresponding values. Figure 4.1 declaration includes the equation needed to determine the counts per inches. This is an important value, calculating gear and counts per motor are implemented in addition to the diameter of the wheel multiplied by pi. The value is used to do a complete tick/count to get to a certain point. Speed is set to 0.6 and is set after multiple testing. The Elapsed Time was set to runtime, Linear opMode set to myOpMode, and Telemetry set to telemetry. We init () the robot by defining the motors as left_Drive and right_Drive this is also updated in the driver hub software. For the servo, we have declared and named the one servo oneHand, “R2D2” robot has only one hand.



```
5 usages   Senior Capstone Team 1
public class MinibotHardware {
    1 usage
    private  Telemetry telemetry;  //HardwareMinibot robot = new HardwareMinibot(); // uses Minibot Hardware

    /* Declare OpMode members. */
    13 usages
    private LinearOpMode myOpMode = null;   // gain access to methods in the calling OpMode.

    // Define Motor and Servo objects  (Make them private so they can't be accessed externally)
    13 usages
    private DcMotor leftDrive   = null; //LEFT FRONT MOTOR
    13 usages
    private DcMotor rightDrive  = null;// RIGHT FRONT MOTOR
    3 usages
    private Servo   oneHand = null; // servo restrain

    // Define Drive constants.  Make them public so they CAN be used by the calling OpMode
    2 usages
    public static final double MID_SERVO       =  0.5 ;
    no usages
    public static final double HAND_SPEED      =  0.02 ;  // sets rate to move servo
    1 usage
    static final double COUNTS_PER_MOTOR_REV = 1440;    // eg: TETRIX Motor Encoder
    1 usage
    static final double DRIVE_GEAR_REDUCTION = 1.0;     // No External Gearing.
    1 usage
    static final double WHEEL_DIAMETER_INCHES = 4.0;     // For figuring circumference
    2 usages
    static final double COUNTS_PER_INCH = (COUNTS_PER_MOTOR_REV * DRIVE_GEAR_REDUCTION) /
            (WHEEL_DIAMETER_INCHES * 3.1415);
    no usages
    static final double     DRIVE_SPEED             = 0.6;
    no usages
    static final double     TURN_SPEED              = 0.5;
   // TouchSensor touchSensor;  // Touch sensor Object
    //NormalizedColorSensor colorSensor;
    //View relativeLayout;
    //HardwareMap hwMap = null;
    3 usages
    private ElapsedTime runtime;
      // Define a constructor that allows the OpMode to pass a reference to itself.
    3 usages   Senior Capstone Team 1
    public MinibotHardware(LinearOpMode opMode, Telemetry telemetry) {
        myOpMode= opMode;
        this.telemetry = telemetry;
        runtime = new ElapsedTime();
    }
```

Figure 4.1 MinibotHardware.java image 1

Figure 4.2 is another portion of our hardware code. The methods start with drive Robot, which is set to Drive and Turn which combines left and right to perform a blended motion. Left and right are set as follows: double left = Drive + Turn; double right = Drive - Turn; adding scale of +/- 1.0 by coding an if statement. This is key to how we implement the encoder in the movement when reversing turning and moving forward. A setDrivePower method was also implemented to output the values of the motor drives. This was set to left Wheel and rightWheel values. The setHandPosition method implemented an offset of 0.5 min/max and labeled the hand as oneHand. (This was sent in the driver hub to identify the hand.)



```
5 usages   Senior Capstone Team 1
public class MinibotHardware {
    1 usage
    private  Telemetry telemetry;  //HardwareMinibot robot = new HardwareMinibot(); // uses Minibot Hardware

    /* Declare OpMode members. */
    13 usages
    private LinearOpMode myOpMode = null;   // gain access to methods in the calling OpMode.

    // Define Motor and Servo objects  (Make them private so they can't be accessed externally)
    13 usages
    private DcMotor leftDrive   = null; //LEFT FRONT MOTOR
    13 usages
    private DcMotor rightDrive  = null;// RIGHT FRONT MOTOR
    3 usages
    private Servo   oneHand = null; // servo restrain

    // Define Drive constants.  Make them public so they CAN be used by the calling OpMode
    2 usages
    public static final double MID_SERVO       =  0.5 ;
    no usages
    public static final double HAND_SPEED      =  0.02 ;  // sets rate to move servo
    1 usage
    static final double COUNTS_PER_MOTOR_REV = 1440;    // eg: TETRIX Motor Encoder
    1 usage
    static final double DRIVE_GEAR_REDUCTION = 1.0;     // No External Gearing.
    1 usage
    static final double WHEEL_DIAMETER_INCHES = 4.0;     // For figuring circumference
    2 usages
    static final double COUNTS_PER_INCH = (COUNTS_PER_MOTOR_REV * DRIVE_GEAR_REDUCTION) /
            (WHEEL_DIAMETER_INCHES * 3.1415);
    no usages
    static final double     DRIVE_SPEED             = 0.6;
    no usages
    static final double     TURN_SPEED              = 0.5;
   // TouchSensor touchSensor;  // Touch sensor Object
    //NormalizedColorSensor colorSensor;
    //View relativeLayout;
    //HardwareMap hwMap = null;
    3 usages
    private ElapsedTime runtime;
      // Define a constructor that allows the OpMode to pass a reference to itself.
    3 usages   Senior Capstone Team 1
    public MinibotHardware(LinearOpMode opMode, Telemetry telemetry) {
        myOpMode= opMode;
        this.telemetry = telemetry;
        runtime = new ElapsedTime();
    }
```

Figure 4.2 MinibotHardware.java image 2

Figure 4.3 code includes the encoder Drive method with values set at double speed, double left Inches, double right Inches, and double timeouts. The speed, inches, and timeout encoder values are very important in how to manipulate the robot in the autonomous section of the game. The values are set in the file that will call this method and set the setting for reversal, forward drive, and turns. In this method, a new target position is coded and passed to the motor controller. The RUN _TO_POSITION was activated followed by a timeout and start motion. This aids in adding a pause after an action to allow the robot to perform its next action correctly. The method also includes a while loop while the robot is still active, and time is left for the motors to run. This code indicates that when the target is met the robot will stop. Lastly in the image below the stop-all motion and turn-off position are also added indicating to the robot to go and park in a designated area in the backstage spike marked red.

```
13 usages   Senior Capstone Team 1
public void encoderDrive(double speed, double leftInches, double rightInches, double timeoutS) {

    int newLeftTarget;
    int newRightTarget;

    // Ensure that the OpMode is still active
    if (myOpMode.opModeIsActive()) {

        // Determine new target position, and pass to motor controller
        newLeftTarget = leftDrive.getCurrentPosition() + (int) (leftInches * COUNTS_PER_INCH);
        newRightTarget = rightDrive.getCurrentPosition() + (int) (rightInches * COUNTS_PER_INCH);
        leftDrive.setTargetPosition(newLeftTarget);
        rightDrive.setTargetPosition(newRightTarget);

        // Turn On RUN_TO_POSITION
        leftDrive.setMode(DcMotor.RunMode.RUN_TO_POSITION);
        rightDrive.setMode(DcMotor.RunMode.RUN_TO_POSITION);

        // reset the timeout time and start motion.
        runtime.reset();
        leftDrive.setPower(Math.abs(speed));
        rightDrive.setPower(Math.abs(speed));

        // keep looping while we are still active, and there is time left, and both motors are running.
        // Note: We use (isBusy() && isBusy()) in the loop test, which means that when EITHER motor hits
        // its target position, the motion will stop.  This is "safer" in the event that the robot will
        // always end the motion as soon as possible.
        // However, if you require that BOTH motors have finished their moves before the robot continues
        // onto the next step, use (isBusy() || isBusy()) in the loop test.
        while ((myOpMode.opModeIsActive() &&
                (runtime.seconds() < timeoutS) &&
                (leftDrive.isBusy() && rightDrive.isBusy()))) {

            // Display it for the driver.
            myOpMode.telemetry.addData( caption: "Running to",  format: " %7d :%7d", newLeftTarget, newRightTarget);
            myOpMode.telemetry.addData( caption: "Currently at",  format: " at %7d :%7d",
                    leftDrive.getCurrentPosition(), rightDrive.getCurrentPosition());
            myOpMode.telemetry.update();
        }

        // Stop all motion;
        leftDrive.setPower(0);
        rightDrive.setPower(0);

        // Turn off RUN_TO_POSITION
        leftDrive.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
        rightDrive.setMode(DcMotor.RunMode.RUN_USING_ENCODER);


        myOpMode.telemetry.addData( caption: "Path",  value: "Complete");
```

3.1 Figure:4.3 MinibotHardware.java image 3

3.2 4.1 Application Information flows - Autonomous phase

3.3 In the autonomous phase of a match, the robot operates without any human input or control.

Team One is red and will operate in the top section near backstage. The tiles on the field are 24W X 24H inches with 6 tiles lined down and 6 across. This is an important measurement since it plays a role in the encoder coding, we use in our robot's code. The autonomous portion of the game is a 30-second timestamp to get from one point to another scoring the most points. Our code has implemented three solutions ready to score the most possible points after randomization is determined. Autonomous code incorporated the April tags to determine what action the robot will implement. We added a while loop with three options depending on what April tag is read the while loop will be implemented.

The **center** is one of the options after randomization is completed layout in below Figure 4.1.1 The April tag indicates randomization object is in the center spike. The robot will approach the red spike in the center 3-C. Starting in tile 4-C in the center tile facing the spike the robot will continue for 2.5s. The pixel will be placed on the center spike. The robot will reverse back to 4-C, turn right for 1.15 seconds with stops added in between each action. The robot will then continue straight toward tile 4-A for 3.5s and make a left turn to proceed to 3-A, a right turn on 3-A in front of the backdrop located on the backstage red spike mark. Backstage is another point where the robot will use April Tag and drop pixel points are added if it centers with the object on the spike. Lastly, the robot will turn right for 1.15 s and continue 4-A robot will park to obtain additional points before the 30 seconds expiration

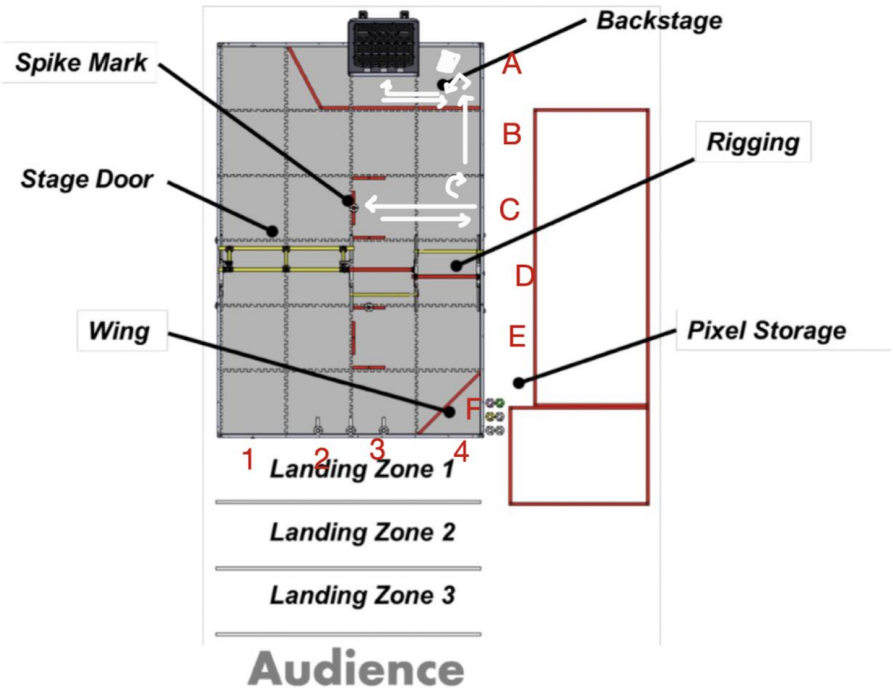


Figure 4.1.1 Center

The while loop has an if-else option if the randomization cone lands on the **left** side see Figure 4.1.2. The April tag indicates randomization object is in the left spike the robot will approach the red spike in the center 3-C. Starting in tile 4-C in the center tile facing the spike the robot will continue for 2.5s. Turn left in tile 3-C for 1.15s and drop the pixel. The robot turns left again to face the forward towards 4-C. The robot will then continue straight toward tile 4-A for 3.5s and make a left turn to proceed to 3-A, a right turn on 3-A in front of the backdrop located on the backstage red spike mark. Backstage is another point where the robot will use April Tag and drop pixel points are added if it centers with the object on the spike. Lastly, the robot will turn right for 1.15 s and continue 4-A robot will park to obtain additional points before the 30 seconds expiration

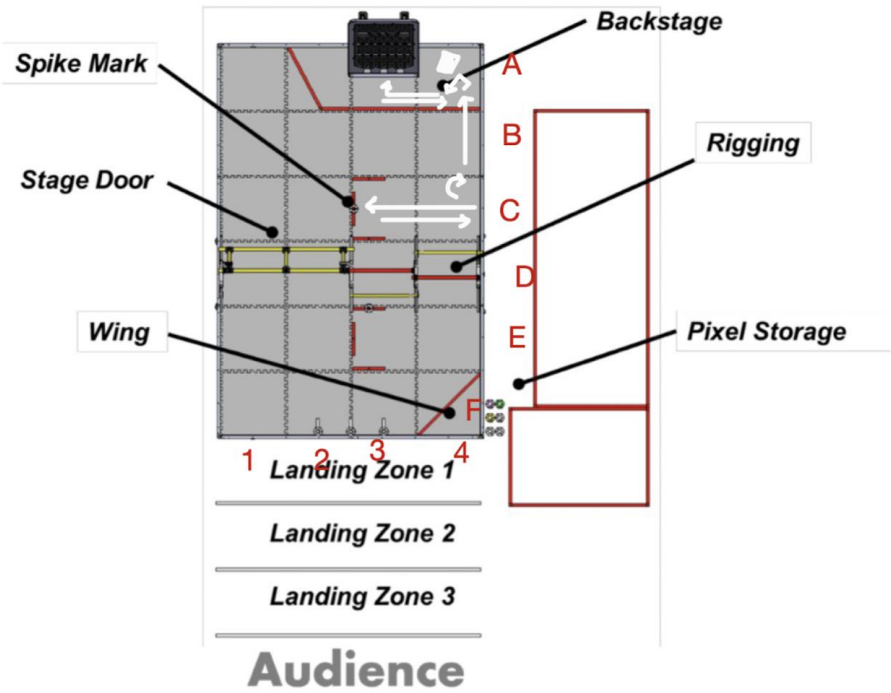


Figure 4.1.2 Left

The last option in the else for the while loop is if the randomization selected is on the **right** side see figure 4.1.3. The April tag indicates randomization object is in the right spike the robot will approach the red spike in the center 3-C. Starting in tile 4-C in the center tile facing the spike the robot will continue for 2.5s. Turn right into tile 3-C for 1.15s and drop the pixel. The robot turns right for 1.15s again to face forward toward 4-C. The robot will then continue straight toward tile 4-A for 3.5s and make a left turn to proceed to 3-A, a right turn on 3-A in front of the backdrop located on the backstage red spike mark. Backstage is another point where the robot will use April Tag and drop pixel points are added if it centers with the object on the spike. Lastly, the robot will turn right for 1.15 s and continue 4-A robot will park to obtain additional points before the 30 seconds expiration

The code for April tags and adding the camera to the hardware was included in the code. The April tags were declared and tested so that the object tag was detected, Modifications were done to the speed of the robot. Due to hardware under the tiles, the field speed was manipulated to the right wheel. The physics of the field changed and was causing issues for the robot and its equilibrium. The autonomous portion of the competition includes a camera that was implemented in the code. The camera is used to read the April tag and instruct the robot what action should be taken. The robot is still in process and should be tested until competition day.

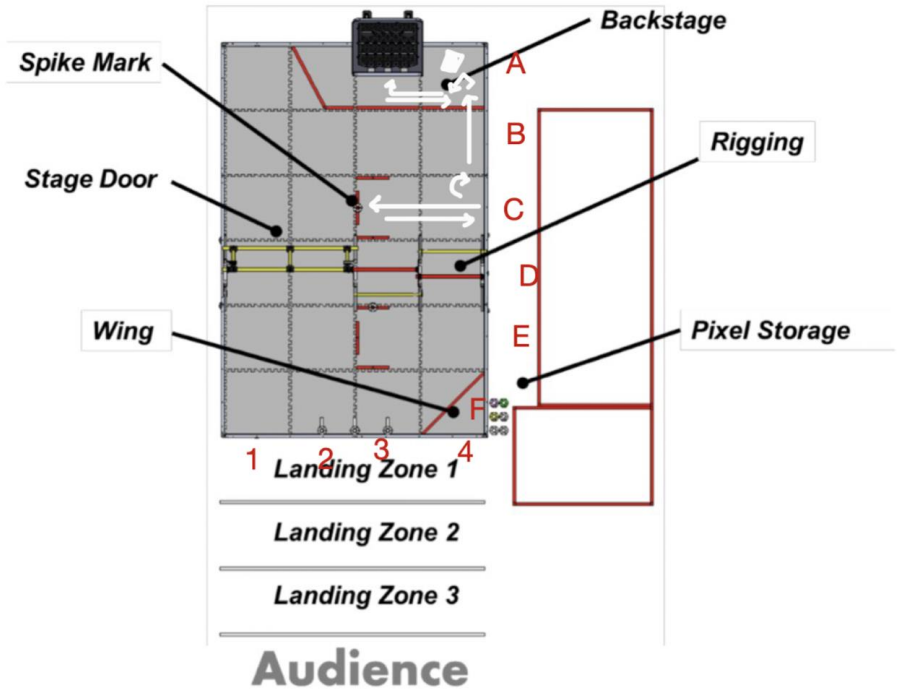


Figure 4.1.3 Right

April tags are like a 2D barcode or a simplified QR Code. It contains a number ID code and can be used for location and orientation. FIRST uses a 36h11 family tag containing 10 X 10 pixels. The small black and white squares are called pixels and have an outer white and inner black color to them. The new SDK l provides position and orientation for the camera's view.

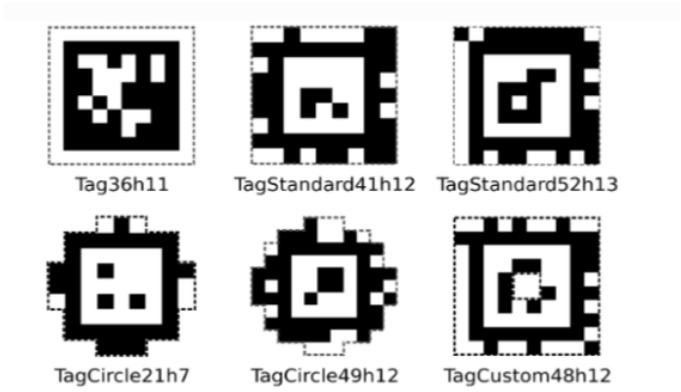


Figure 4.1.3 Sample of different April Tag families

Figure 4.1.4 April Tag Recognition in Robot Driver Station

The camera image above is from a Robot Controller device. The colors indicate that: the 3-dimensional path is single and indicated by the green star. You can see that the center of the April Tag in a yellow star, translation offset has three traditional components labeled as follows: X distance (horizontal orange line) is from the center to the right, Y distance (not shown) is from the lens center, outwards, Z distance (vertical orange line) is from the center, upwards. Navigation is best done with continuous OpMode "**while () loop**" should regularly read the updated pose data, to guide the robot's driving actions.

To detect the April Tag, we printed a sleeve paper to wrap around a red cone. We used OpenCV to detect the Aril tag code. The code declares a name for each April tag and when the camera detects the code the while loop with initiate. The April tags are sensitive to light if not enough the camera will not read them properly and have some misalignments

4.2 Teleop

The code for the teleop is manipulated by using an LG controller. The remote control was labeled based on what each button on the remote was to do. The code has a left and right stick set to y and x. The gamepad hand position was also coded to react when the gamepad presses the buttons assigned. This portion of the game will be manipulated by an assigned team member. The code assigned to the robot is to be able to manipulate and drive the robot. Speed is manipulated so that we can accomplish the best movement to obtain points.

Figure 4.2.1 Game Pad

Figure 4.2.1 MinibotTeleop.java

Figure 4.2.2 MinibotTeleop.java

4.3 End Game

This section is at the end and allows us the chance to obtain the most points. The points are achieved by driving our robot to the 3-F location on the field. The robot arm will launch the drone that is created by

using paper. The landing zone is located in front of section F. Our goal is to have a lightweight paper plane drone that can achieve landing in Zone 3 for the max points.

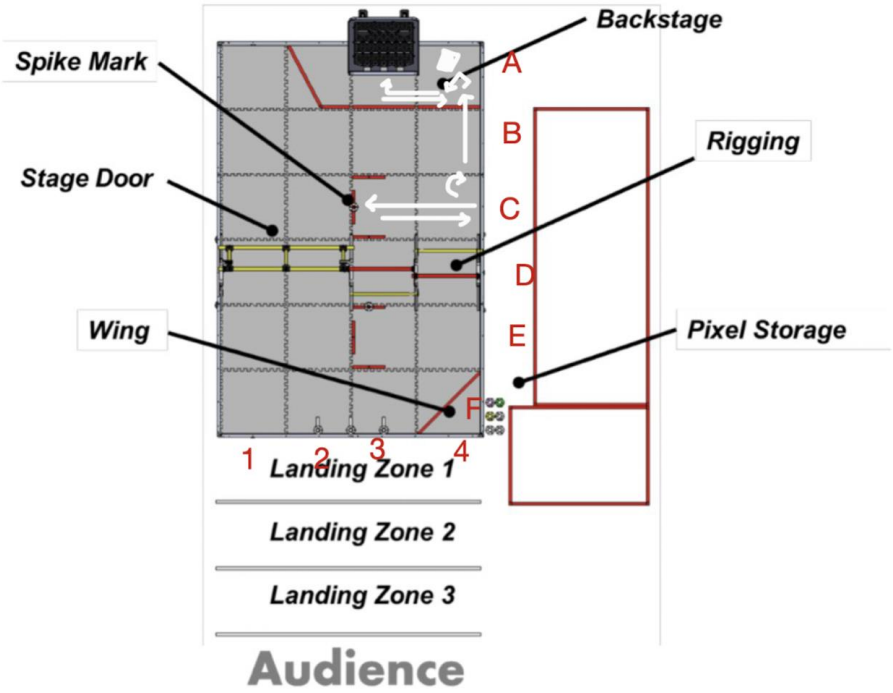


Figure 4.3.1 Endgame

4 5. Capabilities===

Team 1 wrote three Java classes: MinibotEncoder.java, MinibotHardware.java, MinibotTeleop.java

"Op modes" (which stands for "operational modes") is to specify the behavior of a robot. *Op modes* are computer programs that are used to customize the behavior of a competition robot. The Robot Controller can *execute* a selected op mode to perform certain tasks during a match Below is description of our R2D2 Opmodes:

For Autonomous Period, we are using MinibotEncoder.java program, it is used to determine position of the cone using the April Tags And then, select the path to move safely without crashing to other robots towards the Tile A4 and park.

MinibotHardware.java: Using this Opmode Our R2D2 can move Forward, Backward, Right turn, Left turn. Based on the Value of Counts per inch and it counts in every full turn of the wheel.

MinibotTeleop.java: This Op mode is used while controlling the robot manually using the gamepad. We have coded the gamepad keys to Move the robot according to our instructions on the gamepad.

We made sure our robot won't collide with another team's robot, to do that we made our robot is Faster than other robots and We stayed along the Column 4 which no other team is using.

6. *Definitions and Acronyms*

- **OpMode**: OpMode or Operational Mode (also Op Mode and opmode) is a class in the FTC SDK (robot controller app source code). This class is used to add your code to the controller app. Your code is actually only a subset of the controller app, with the FTC source code providing the remainder.

- **TeleOp:** Teleoperation is the term used to describe directing a robot from a distance, like how television and telephone refer to distance vision and distance voice. This contrasts with autonomous operation, in which the robot is assigned a task and then goes off to execute it on its own.

- **Autonomous**: Being self-sufficient and able to make your own decisions. An autonomous organization, country, or area is self-governing and self-sufficient.

- **Android Studio** – **A.S**: Android Studio is Google's official integrated development environment for the Android operating system. It is based on JetBrains' IntelliJ IDEA software and is specifically developed for Android programming. It can be downloaded for Windows, macOS, and Linux-based operating systems.

- **FTC:** First Tech Challenge

- **SDK:** Software Development Kit

- **Seconds – s:** The second (s or sec) is the International System of Units (SI) unit of time measurement.

5 7. Requirements

It is of vital importance that the Driver Station App be updated to a version that meets or exceeds the minimum Driver Station App.

FIRST requires the hardware platform used in the FIRST Tech Challenge to be the REV Control Hub and REV Driver Hub. Teams use computers for software development; Windows Performance Laptop as well as supporting MacOS Standard Laptop. REV Hardware Client is not supported on MacOS. Must have an active internet connection during software development. Access to https://github.com is required by the REV Hardware Client to download and install required season software updates and is required for Android Studio users to download software templates.

8. Open Issues

We do not have an arm to pick up the pixel or scoop, we need to mimic the action.

The tiles are now including other parts of the field riggings. The field layout has changed and there are areas of the surface that are not straight and smooth.

Other alliance teams are not complete with the left and right movements to test both robots together and correct any issues.

9. Acknowledgements

Thank You! Governors State University for allowing us to do this as a Senior Capstone. Thank you! Professor X. Tang for providing the Robotic equipment to complete this project!

10. References

FIRST Tech Challenge Programming Resources: Aug. 22, 2023

https://www.firstinspires.org/resource-library/ftc/technology-information-and-resources

CENTERSTAGE Game Animation 2023-2024 FIRST Tech Challenge: 9 Sept 2023

https://youtu.be/lDcZCR4GOpY?si=UKxw_SXDM_Gn2vlB

OpenCV: https://github.com/OpenFTC/EasyOpenCV

Application Architecture: https://www.firstinspires.org/resource-library/ftc/technology-information-and-resources

Informational video on servo: https://youtu.be/1WnGv-DPexc?si=oHTw5AgkhDYvnnvV

Informational information related to Robot Hardware: https://www.revrobotics.com/ftc/motion/motors-servos/

Java code tutorials: https://www.w3schools.com/

Android Studio Tutorials for MacOS and Windows: https://developer.android.com/studio?gclid=CjwKCAiA0sygBhBxEiwAeNx9N6oQqAcKmZitZtMB4Bj-emmXZITdki10Dk7Vu6ctAGmZOsW_1Wk9-hoCBgQQAvD_BwE&gclsrc=aw.ds

9. GIT tutorial: https://git-scm.com/downloads

10. Git and GitHub for Beginners - Crash Course:

https://youtu.be/RGOj5yH7evk? si=m5bVb17mu-MXwflS

11. Appendices

Link to code Text is available in the GitHub Link below:

https://github.com/***sgarciaabad***/FtcRobotController-9.0.git